

Lab 05 — Ethereum accounts, gas & the EVM (read-only)

Bài thực hành 05 — Tài khoản, gas & EVM của Ethereum (chỉ đọc)

Time / Thời lượng: 50 minutes in class / 50 phút trên lớp. **Network / Mạng:** TrustKeys L1 testnet — RPC <https://l1testnet.trustkeys.network>, chainId **11968** (0x2ec0), EIP-1559 active. Fallback / dự phòng: Sepolia + Etherscan. **Pass condition / Điều kiện đạt:** Lab 5.2 prints the base-fee series (all CHECK = OK) **and** you demo Lab 5.3 (a real type-2 tx decomposed into burn + tip) to the TA. / Lab 5.2 in được chuỗi base fee (mọi CHECK = OK) **và** demo Lab 5.3 (một giao dịch type-2 thật, phân rã thành đốt + tip) cho trợ giảng.

This lab is read-only web3.py + MetaMask — no contract deployment. Contract dev starts Session 6 (Hardhat + Remix). *Bài này chỉ đọc bằng web3.py + MetaMask — không triển khai hợp đồng; lập trình hợp đồng bắt đầu từ Buổi 6 (Hardhat + Remix).*

Setup / Chuẩn bị:

```
python3 --version # >= 3.8
pip install web3 # web3.py v6 or v7 / ba'n 6 hoặc 7
```

Submit / Nộp: one file lab05_<your-name>.md with Q1–Q6 answers + the two screenshots from Lab 5.4. / một file lab05_<ten>.md gồm câu trả lời Q1–Q6 + hai ảnh chụp màn hình ở Lab 5.4.

Lab 5.1 — Add TrustKeys L1 to MetaMask (10')

EN: MetaMask → network selector → **Add a network manually**, then fill in:

Field	Value
Network name	TrustKeys L1 testnet
New RPC URL	https://l1testnet.trustkeys.network
Chain ID	11968
Currency symbol	coin nền tảng TrustKeys (<i>instructor confirms in class</i>)
Block explorer	(<i>leave blank — none yet</i>)

VI: MetaMask → chọn mạng → **Thêm mạng thủ công**, điền bảng trên. Ký hiệu đồng coin do giảng viên xác nhận trên lớp; để trống ô block explorer.

Because your wallet was derived from a BIP39 mnemonic (Session 1/3, BIP44 path m/44'/60'/0'/0/0), the **same address** appears here as in your app.dem.chat wallet. *Vì ví*

dẫn xuất từ mnemonic BIP39 (Buổi 1/3, đường dẫn BIP44), địa chỉ ở đây trùng với ví `app.dem.chat` của bạn.

CHECK: the account address in MetaMask on TrustKeys L1 == the address you used in Session 1. / Địa chỉ trong MetaMask trên TrustKeys L1 trùng địa chỉ bạn đã dùng ở Buổi 1.

Q1 / Câu hỏi 1: Adding a network only changes the RPC endpoint + chainId — your private key never leaves the device. Why is chainId *also* part of every signed transaction (recall EIP-155, Session 3)? / Thêm mạng chỉ đổi endpoint RPC + chainId — khóa bí mật không rời thiết bị. Vì sao chainId *cũng* nằm trong mọi giao dịch được ký (nhớ EIP-155, Buổi 3)?

Lab 5.2 — Read the chain with web3.py (15')

EN: Run `solutions/fee_history.py` (or type it yourself). It connects read-only, then calls `eth_getBalance`, `eth_getBlockByNumber`, and `eth_feeHistory`, and prints the last 20 base fees. **VI:** Chạy `solutions/fee_history.py` (hoặc tự gõ). Kết nối chỉ-đọc rồi gọi `eth_getBalance`, `eth_getBlockByNumber`, `eth_feeHistory`, in 20 base fee gần nhất.

Minimal core / Phần lõi tối thiểu:

```
from web3 import Web3
from web3.middleware import ExtraDataToPOAMiddleware # TrustKeys is a PoA (clique) chain

w3 = Web3(Web3.HTTPProvider("https://l1testnet.trustkeys.network"))
w3.middleware_onion.inject(ExtraDataToPOAMiddleware, layer=0) # else ExtraDataLengthError

blk = w3.eth.get_block("latest")
print("baseFeePerGas =", blk["baseFeePerGas"], "wei")

fh = w3.eth.fee_history(20, "latest", [10, 50, 90]) # [tip percentiles]
for i, base in enumerate(fh["baseFeePerGas"][:-1]):
    print(fh["oldestBlock"] + i, base, fh["gasUsedRatio"][i])
```

Gotcha / Lưu ý: TrustKeys L1 is a **PoA (clique)** geth chain, so `extraData` is 97 bytes, not 32. Without `ExtraDataToPOAMiddleware` you get `ExtraDataLengthError`. This is the same middleware you need for Sepolia-era testnets and BSC. / TrustKeys L1 là chuỗi PoA (clique), `extraData` dài 97 byte nên phải nạp middleware, nếu không sẽ lỗi.

CHECK: you see 20 lines of base fees + a "chain is BUSY/quiet" verdict. / Bạn thấy 20 dòng base fee + kết luận chuỗi bận/rảnh.

Q2 / Câu hỏi 2: On TrustKeys L1 right now the base fee is only a handful of wei (well below 1 gwei) and gasUsedRatio is ~2%. Using the EIP-1559 rule (target 15M / cap 30M gas; $\pm 12.5\%$ per block), explain *why* the base fee stays at the floor. / Base fee hiện chỉ vài wei (dưới 1 gwei), gasUsedRatio ~2%. Dùng quy tắc EIP-1559 (mục tiêu 15M / trần 30M gas; $\pm 12.5\%$ /block), giải thích vì sao base fee đứng ở đáy.

Q3 / Câu hỏi 3: eth_feeHistory returns baseFeePerGas of length N+1 but gasUsedRatio of length N. What is that extra element, and how would a wallet use it to set maxFeePerGas? / eth_feeHistory trả baseFeePerGas dài N+1 nhưng gasUsedRatio dài N. Phần tử dư là gì, và ví dùng nó để đặt maxFeePerGas thế nào?

Lab 5.3 — Send a type-2 tx and decompose the fee (15')

EN: 1. In MetaMask (TrustKeys L1) send **0.001** coin to a classmate (or to yourself). Confirm it is a **type-2 (EIP-1559)** tx — MetaMask shows *max base fee + priority fee*, not a single gas price. 2. Copy the **transaction hash**. 3. Fetch the receipt over RPC and decompose the fee:

VI: 1. Trong MetaMask (TrustKeys L1) gửi **0.001** coin cho bạn cùng lớp (hoặc cho chính mình). Xác nhận đây là giao dịch **type-2 (EIP-1559)**. 2. Sao chép **hash giao dịch**. 3. Lấy receipt qua RPC và phân rã phí:

```
h          =          "0x...your          tx          hash..."
rcpt              =          w3.eth.get_transaction_receipt(h)
tx              =          w3.eth.get_transaction(h)
blk              =          w3.eth.get_block(rcpt["blockNumber"])

base_fee              =          blk["baseFeePerGas"]
gas_used              =          rcpt["gasUsed"]
eff_price = rcpt["effectiveGasPrice"]          # baseFee + min(tip, maxFee -
baseFee)

paid              =          gas_used          *          eff_price
burned = gas_used * base_fee          # leaves circulation / rời lưu
thông
tip    = gas_used * (eff_price - base_fee)          # to the block proposer / về
proposer
print(paid, "=", burned, "+", tip, "=", paid == burned + tip)
```

The identity **paid = burned + tip** always holds. On this testnet the base fee is tiny, so almost the whole fee is *tip*; on Ethereum mainnet during congestion the *burn* dominates. *Đẳng thức luôn đúng; ở testnet base fee bé nên phí gần như toàn là tip, còn trên mainnet lúc nghẽn thì phần đốt chiếm phần lớn.*

CHECK / demo: show the TA paid == burned + tip for your real tx. / Cho trợ giảng xem paid == burned + tip với giao dịch thật của bạn.

Q4 / Câu hỏi 4: Your effectiveGasPrice equals baseFee + min(maxPriorityFee, maxFee - baseFee). If you had raised your MetaMask *max base fee* setting but left the *priority fee* the same, would effectiveGasPrice change? Why / why not? / Nếu bạn tăng *max base fee* trong MetaMask nhưng giữ nguyên *priority fee*, effectiveGasPrice có đổi không? Vì sao?

Q5 / Câu hỏi 5: A simple transfer used exactly **21,000** gas. Where does that number come from, and why does even a *failed* transfer still cost you gas? / Một giao dịch chuyển đơn giản dùng đúng **21.000** gas. Con số đó từ đâu ra, và vì sao một giao dịch *thất bại* vẫn tốn gas?

Lab 5.4 — Trace a contract interaction on Sepolia Etherscan (10')

TrustKeys L1 has no block explorer yet, so we borrow Sepolia's for the "explorer experience". *TrustKeys L1 chưa có explorer, nên mượn của Sepolia.*

EN: 1. Open <https://sepolia.etherscan.io> and pick any **contract** transaction (e.g. search a known token, open a recent tx that calls a function — not a plain ETH transfer). 2. On the tx page, expand "**Click to see More**" → find **Input Data** and press **Decode Input Data**. Note the 4-byte **function selector** and the decoded arguments. 3. Open the **Logs** tab and read one emitted event (e.g. Transfer(address,address,uint256)).

VI: 1. Mở <https://sepolia.etherscan.io>, chọn một giao dịch **gọi hợp đồng** (không phải chuyển ETH thường). 2. Búng "**Click to see More**" → **Input Data** → **Decode Input Data**. Ghi lại **selector 4 byte** và các tham số đã giải mã. 3. Mở tab **Logs**, đọc một sự kiện phát ra.

Deliverable / Sản phẩm: two screenshots in your submission — (a) decoded Input Data showing the selector + args, (b) the Logs tab. / Hai ảnh chụp trong bài nộp: (a) Input Data đã giải mã kèm selector + tham số, (b) tab Logs.

Q6 / Câu hỏi 6: The selector is keccak256("transfer(address,uint256)")[:4] = 0xa9059cbb. Explain in two sentences how the EVM uses these 4 bytes to jump to the right function (the "dispatcher"), and why Etherscan needs the contract's **ABI** to decode the rest. / Selector là 4 byte đầu của keccak của chữ ký hàm. Giải thích trong 2 câu EVM dùng 4 byte này để nhảy tới đúng hàm (dispatcher) thế nào, và vì sao Etherscan cần **ABI** của hợp đồng để giải mã phần còn lại.

Homework / Bài về nhà (trước Buổi 6 / before Session 6)

1. Run `fee_history.py` at **two different hours** and compare the base-fee series on **TrustKeys L1 vs Sepolia** (set a public Sepolia RPC such as <https://ethereum-sepolia-rpc.publicnode.com> — remember the PoA middleware there too). ½ page: why do they differ? / Chạy 2 thời điểm, so sánh TrustKeys L1 vs Sepolia — nửa trang: vì sao khác?
 2. On **evm.codes** find the gas cost of ADD, SLOAD (cold vs warm), SSTORE (0→x). Explain the ratio in 3 sentences. / Tra giá gas trên evm.codes, giải thích tỷ lệ trong 3 câu.
 3. **Seminar (midterm Part B):** register your group (3–4) + top-3 topic choices **before end of this week** — see the seminar handout / class channel. / Đăng ký nhóm + 3 nguyện vọng đề tài trước cuối tuần này.
 4. Read *Mastering Ethereum* ch. 13 (EVM) and the ethereum.org PoS pages. / Đọc Mastering Ethereum chương 13 và trang PoS của ethereum.org.
-

Reference output / Kết quả tham chiếu

Real output from `solutions/fee_history.py` against the live TrustKeys L1 RPC (captured 2026-08-14, head block 717,152). Your numbers will differ. / Kết quả thật khi chạy trên RPC TrustKeys L1 (ngày 2026-08-14, block 717.152); số của bạn sẽ khác.

Connected to TrustKeys L1 chainId=11968 head block=717152

```
[eth_getBlockByNumber                                     latest]
      number                                             =      717152
      gasLimit                                           =      30,000,000
      gasUsed                                           =      173,386      (0.58% full)
baseFeePerGas= 8      (0.0000000008 gwei (8 wei)) <- EIP-1559 active
      #txs                                              =      1
```

```
[eth_getBalance]
      address      =      0x5A351A9280e2e5b9324408EC5769D0745d7aB512
balance = 1935177932000000000 wei = 1.935177932 coin
```

```
[eth_feeHistory]      last 20 blocks (base fee series)
      block      baseFee(gwei)      gasUsedRatio      tip p50 (gwei)
      717133      0.0000      7.96%      ...
      717152      0.0000      0.58%      2.0000
      next block's projected baseFee = 0.0000000008 gwei (8 wei)
      average gasUsedRatio over window = 1.89% -> chain is quiet
```

```
[fee decomposition for a 21,000-gas transfer] (illustrative caps)
      baseFee                                           =      8 wei
      effectiveGasPrice                                 =      1.0000000008 gwei
paid = 21000000168000 wei = 0.000021000000168 coin
```

```
burned = 168000 wei (leaves circulation)
tip     = 2100000000000000 wei (to proposer)
invariant OK: paid == burned + tip
```